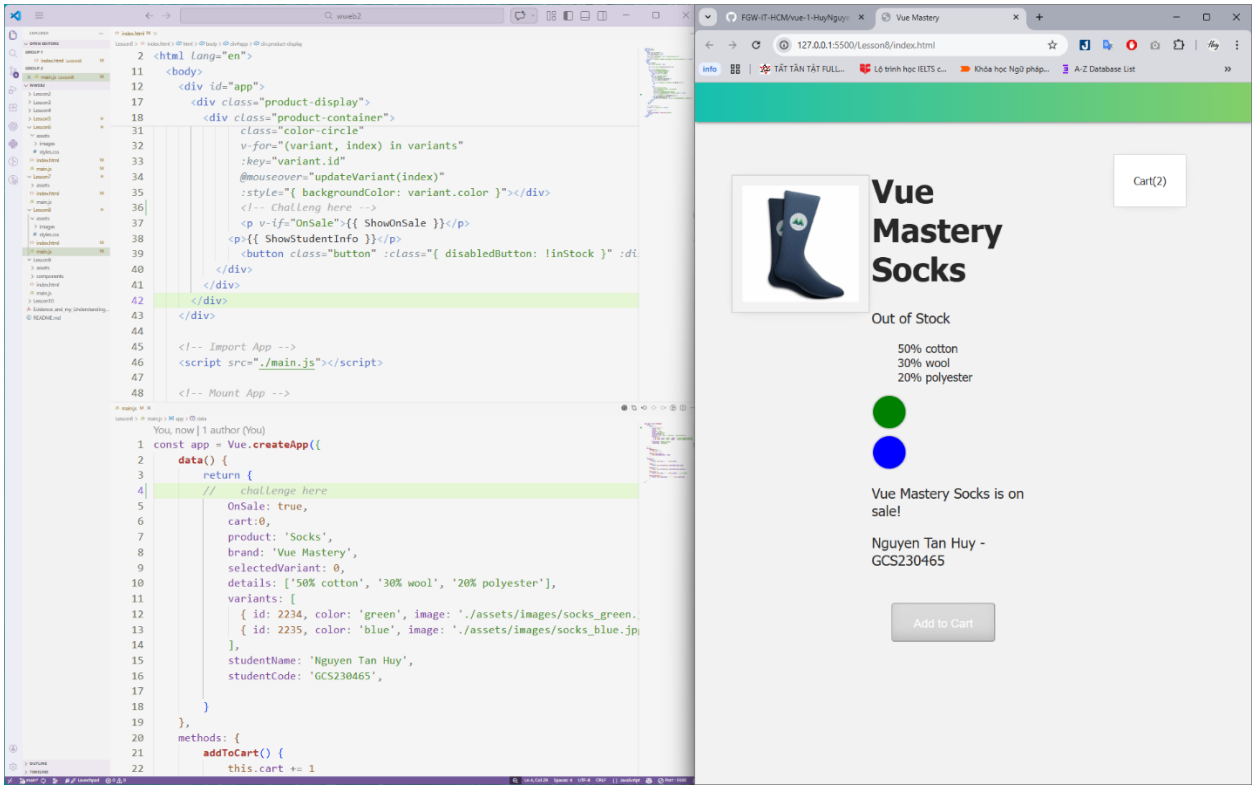
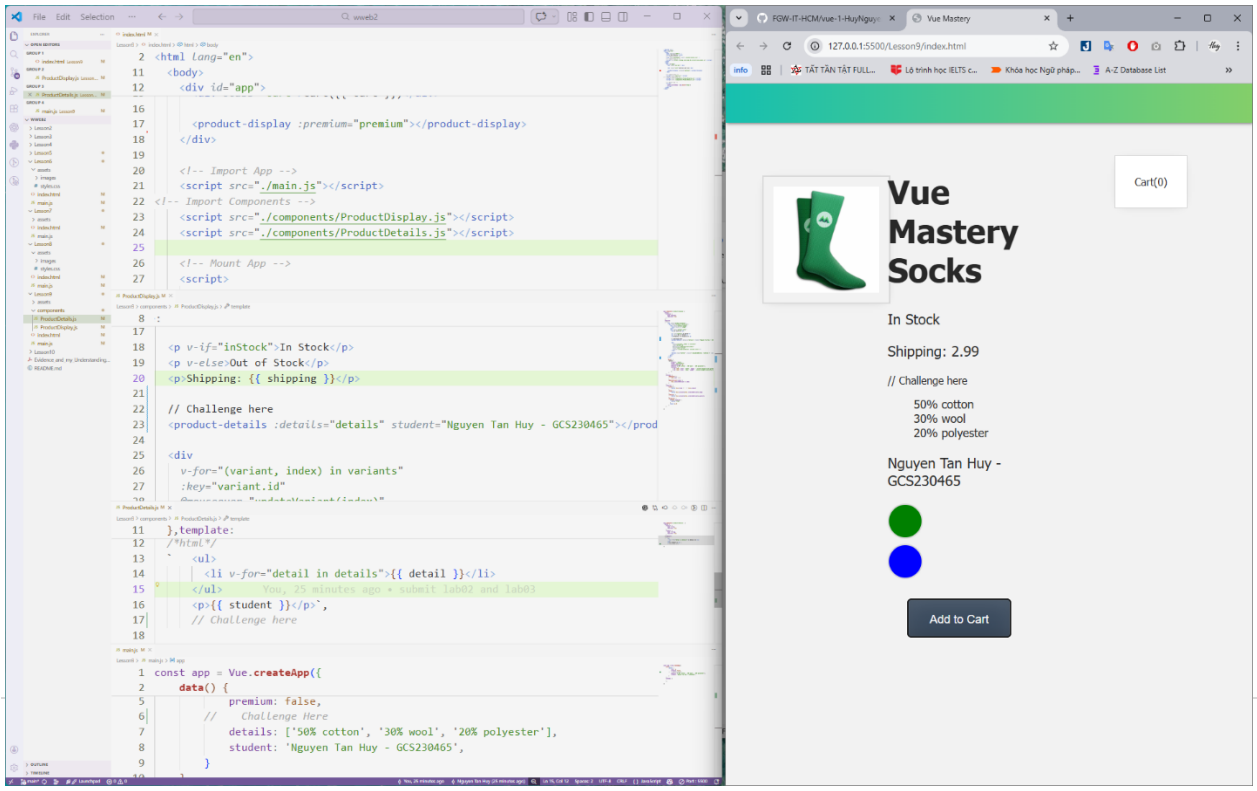
Nguyen Tan Huy – GCS230465

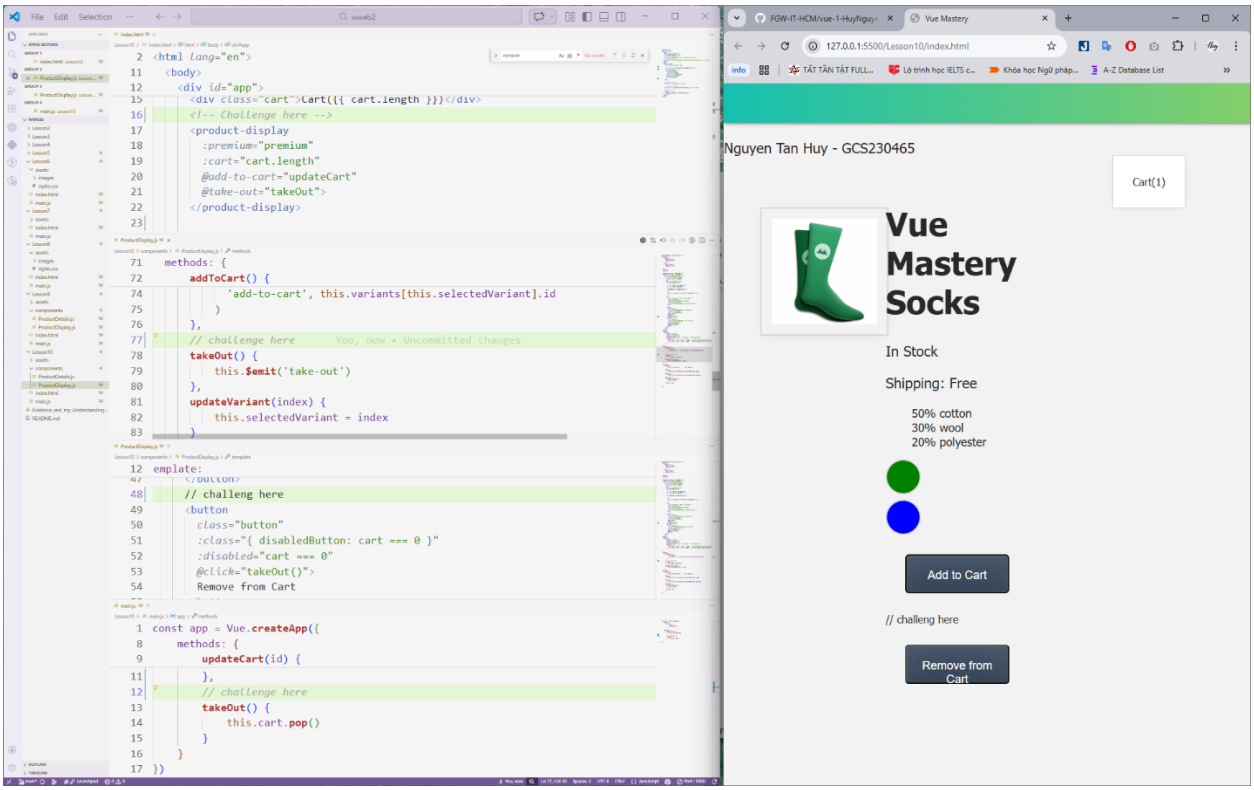
Lab 08-image:



Lab 09-image:



Lab 10-image:


File Edit Selection
wweb2
index.html
<html lang="en">
<body>

Cart({{ cart.length }})
<!-- Challenge here -->
<product-display
:premium="premium"
:cart="cart.length"
@add-to-cart="updateCart"
@take-out="takeOut">
</product-display>
methods: {
addToCart() {
'add-to-cart', this.variants[this.selectedVariant].id
)
},
// challenge here
takeOut() {
this.$emit('take-out')
},
updateVariant(index) {
this.selectedVariant = index
}
emplate:
// challeng here
<button
class="button"
:class="{ disabledButton: cart === 0 }"
:disabled="cart === 0"
@click="takeOut()">
Remove from Cart
const app = Vue.createApp({
methods: {
updateCart(id) {
},
// challenge here
takeOut() {
this.cart.pop()
}
}
})
Vue Mastery
127.0.0.1:5500/Lesson10/index.html
Nguyen Tan Huy - GCS230465
Cart(1)
Vue Mastery Socks
In Stock
Shipping: Free
50% cotton
30% wool
20% polyester
Add to Cart
// challeng here
Remove from Cart

A ~200 words paragraph demonstrating my understanding after completing lessons 8-10 on Vue Mastery:

word count:

After completing **Lab 3**, I started to clearly see how powerful **Vue** really is and where its true strength comes from. Vue does not just simplify JavaScript syntax; it provides a smart way to organize logic, data, and UI together. This lab helped me understand why Vue is widely used in modern front-end development.In **Lesson 8**, I learned about **Computed Properties**. I understood how Vue automatically caches computed values and only recalculates them when their dependencies change. This makes the application more efficient and improves performance. I also learned how to combine multiple variables from the data object into a single computed result, which helps keep the template clean and readable. **Lesson 9** was the most meaningful lesson for me. Creating a component and then reusing it across the application is extremely powerful and saves a lot of development time. I learned how **props** work and how data is passed from a parent component to a child component. This made the structure of Vue applications much clearer and more maintainable. In **Lesson 10**, I learned how **parent and child components communicate** using custom events. The child component uses emit to trigger an event, and the parent component listens to it using @event-name. This two-way communication model helped me fully understand how Vue manages component interaction in a clean and controlled way.